

MASTER_CONTEXT.md

Part 5 — Engineering Standards Summary

Document Purpose

This section defines how Aetheris Trade must be engineered, developed, tested, reviewed, and maintained.

Every developer and AI assistant must follow these standards.

Engineering Philosophy

Every line of code should be:

- Readable
 - Maintainable
 - Scalable
 - Secure
 - Performant
 - Testable
-

Core Engineering Principles

- Simplicity First
 - Reusability First
 - Type Safety Always
 - Security by Design
 - Performance by Default
 - Documentation Before Complexity
-

Official Tech Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

- Framer Motion
-

Backend

- Next.js API Routes
 - Prisma ORM
 - PostgreSQL
-

Authentication

- Clerk
-

AI

- OpenAI API
-

Deployment

- Vercel
-

Project Structure

```
app/  
components/  
features/  
hooks/  
services/  
store/  
lib/  
prisma/  
types/  
utils/  
constants/  
middleware/  
tests/  
docs/
```

Naming Conventions

Folders

kebab-case

Files

kebab-case

Components

PascalCase

Functions

camelCase

Variables

camelCase

Constants

UPPER_SNAKE_CASE

Component Rules

Every component must:

- Have one responsibility
 - Be reusable
 - Be typed
 - Be responsive
 - Follow the Design System
-

State Management

Priority

1. Local State
2. Context API
3. Zustand

Avoid duplicate state.

Data Fetching

Rules

- Prefer Server Components
 - Cache static content
 - Validate responses
 - Handle loading and error states
-

Styling Rules

- Tailwind CSS only
 - No inline styles
 - Use Design Tokens
 - No hardcoded colors
-

Animation Rules

- Framer Motion only
 - 60 FPS target
 - Motion must improve UX
 - Respect reduced-motion settings
-

Backend Rules

Architecture

```
Request
↓
Middleware
↓
```

```
Validation
↓
Service
↓
Repository
↓
Database
```

Business logic must never exist inside UI components.

Database Rules

- Prisma only
 - Migrations only
 - Transactions where required
 - Indexed queries
 - No duplicated data
-

API Rules

- RESTful
 - Versioned
 - Stateless
 - JSON Responses
 - Typed Contracts
-

Git Workflow

Branches

```
main
develop
feature/*
fix/*
hotfix/*
```

Commit Format

```
type(scope): message
```

Example

```
feat(portfolio): add allocation chart
```

Testing Requirements

Every feature requires:

- Unit Tests
- Integration Tests
- UI Testing
- API Testing

Coverage Target

90%+

Security Standards

Mandatory

- Authentication
 - Authorization
 - Input Validation
 - Output Sanitization
 - HTTPS
 - Rate Limiting
 - Secure Environment Variables
-

Performance Targets

Frontend

- Lighthouse ≥ 90

Backend

- API Response $< 500\text{ms}$

Database

- Optimized Queries
-

Documentation Rules

Every feature update requires:

- Documentation Update
 - API Update
 - Architecture Review
 - Changelog Entry
-

Definition of Done

A task is complete only if:

- Feature works
 - Tests pass
 - Documentation updated
 - Build successful
 - Responsive
 - Accessible
 - No TypeScript errors
 - No ESLint errors
-

AI Engineering Rules

Every AI assistant must:

- Follow Engineering Standards
 - Follow Design System
 - Avoid duplicate code
 - Write modular code
 - Preserve scalability
 - Never bypass validation
-

End of Part 5

Next:

- Part 6 — Database & API Blueprint